

Aplicaciones con microprocesadores y microcontroladores.

M. en C. Saul Villegas.

Universidad Autónoma de la Ciudad de México.

13 de septiembre de 2026

Bloque A

Introducción a los sistemas embebidos

Bienvenidos al curso de **Aplicaciones con microprocesadores y microcontroladores**.

Durante el semestre se estudiarán los principios fundamentales para el diseño de sistemas embebidos, integrando *hardware* y *software* para resolver problemas de ingeniería mediante el uso de un microcontrolador moderno.

¿Qué aprenderemos?

Al finalizar el curso el estudiante será capaz de:

- Comprender el funcionamiento de un microcontrolador moderno.
- Desarrollar *firmware* en lenguaje C.
- Integrar sensores, actuadores y módulos de comunicación.
- Diseñar aplicaciones embebidas completas.

Herramientas de desarrollo

Durante el curso se emplearán las siguientes herramientas:

- Raspberry Pi Pico como plataforma experimental.
- Microcontrolador RP2040.
- Lenguaje C.
- Visual Studio Code.
- Pico SDK oficial.
- EasyEDA para el diseño de circuitos impresos.

Más allá de una tarjeta de desarrollo

Filosofía del curso

El objetivo del curso no consiste únicamente en aprender a utilizar una tarjeta de desarrollo. Se busca comprender los principios que permiten diseñar sistemas embebidos que puedan migrarse posteriormente a otras plataformas industriales.

Durante el semestre se estudiarán

- Arquitectura de microcontroladores.
- Programación en C.
- Periféricos.
- Protocolos de comunicación.
- Diseño electrónico.
- Integración de sistemas.

Áreas de aplicación

Los sistemas embebidos forman parte de prácticamente todos los sectores tecnológicos actuales.

- Automatización industrial.
- Telecomunicaciones.
- Instrumentación.
- Internet de las Cosas.
- Robótica.
- Sistemas automotrices.
- Equipos biomédicos.

Objetivo general

Propósito de la asignatura

Al finalizar el curso, el estudiante será capaz de analizar y comprender los conocimientos básicos acerca de los microcontroladores y microprocesadores, su forma de programación e interconexión con diferentes circuitos periféricos para diseñar y construir aplicaciones que permitan la solución de problemas en su área de conocimiento.

Más allá de la programación

Además del desarrollo de *software*, el curso fomenta competencias propias de la ingeniería para abordar problemas reales mediante metodologías de diseño y validación.

- Análisis de problemas.
- Diseño de soluciones.
- Integración de *hardware* y *software*.
- Validación experimental.
- Documentación técnica.

Preparación antes de cada sesión

Antes de asistir a clase, el estudiante deberá revisar el material indicado para familiarizarse con los conceptos que serán abordados durante la sesión.

- Lectura del material recomendado.
- Revisión de hojas de datos.
- Consulta de la documentación oficial.
- Preparación de dudas.

Construcción del conocimiento

Las sesiones teóricas tienen como propósito explicar los fundamentos de los sistemas embebidos, relacionar los conceptos con aplicaciones reales y preparar al estudiante para su implementación en el laboratorio.

Durante la clase

- Se analizarán conceptos fundamentales.
- Se estudiarán ejemplos reales.
- Se resolverán ejercicios.
- Se discutirán buenas prácticas de ingeniería.

Aprendizaje basado en la experimentación

Cada tema desarrollado en clase contará con una práctica, donde el estudiante implementará y verificará experimentalmente los conceptos estudiados.

- Desarrollo de *firmware*.
- Integración de *hardware*.
- Medición y validación.
- Resolución de problemas.

Una competencia profesional

Durante todo el curso se utilizarán documentos técnicos como hojas de datos, manuales de referencia y documentación oficial de Raspberry Pi. El objetivo es que el estudiante desarrolle la capacidad de obtener información directamente de las fuentes del fabricante.

`https://www.raspberrypi.com/documentation/pico-sdk/`

Evaluación del curso

De acuerdo con el programa de estudios, la evaluación del curso se integra por tres etapas:

- Evaluación diagnóstica.
- Evaluaciones formativas.
- Evaluación para certificación.

Cada una tiene un propósito específico para valorar el avance y el logro de las competencias desarrolladas durante el semestre.

Objetivo

Al inicio del curso se aplicará un examen escrito para evaluar los conocimientos previos del estudiante sobre **electrónica digital y comunicaciones analógicas y digitales**, con el propósito de detectar carencias y proponer acciones para corregirlas.

Características

- Examen escrito.
- No forma parte de la calificación final.

Seguimiento del aprendizaje

Durante el desarrollo del curso se dará seguimiento al aprovechamiento del estudiante mediante actividades que permitan verificar la correcta comprensión y aplicación de los conocimientos adquiridos.

Evidencias

- Reportes de prácticas.
- Ejercicios y problemas teóricos.
- Retroalimentación continua.
- Asesorías cuando sean necesarias.

Prácticas

Los reportes deberán entregarse en la siguiente sesión. En caso de detectarse deficiencias, el estudiante podrá repetir la práctica con el objetivo de reforzar los conocimientos y alcanzar los resultados esperados.

Formato de prácticas

Primera modalidad

La evaluación para certificación consta de dos partes:

- Examen teórico con un valor del **60 % al 70 %**.
- Proyecto práctico con un valor del **30 % al 40 %**.

Segunda modalidad

El programa también contempla la integración de un portafolio de evidencias conformado por investigaciones, presentaciones y prácticas desarrolladas durante el semestre.

- Portafolio: **50 % al 60 %**.
- Proyecto o práctica final: porcentaje restante.

Objetivo

Como parte del proceso de aprendizaje, el estudiante desarrollará un proyecto que integre los conocimientos adquiridos durante el semestre, demostrando su capacidad para analizar, diseñar e implementar una aplicación basada en un microcontrolador.

Características del proyecto

Alcance

El proyecto deberá integrar los principales temas abordados durante el curso.

- Desarrollo de *firmware* en lenguaje C.
- Uso de periféricos del microcontrolador.
- Comunicación con dispositivos externos.
- Diseño e integración del *hardware*.

El proyecto no se desarrollará únicamente al final del curso. Cada práctica de laboratorio aportará conocimientos y componentes que posteriormente podrán integrarse en la solución final.

A.2

Sistemas embebidos

¿Qué es un sistema embebido?

Definición

Un sistema embebido (*Embedded System*) es un sistema de cómputo diseñado para realizar una o varias funciones específicas dentro de un dispositivo electrónico.

A diferencia de una computadora de propósito general, un sistema embebido se desarrolla para ejecutar tareas concretas de forma eficiente, confiable y, generalmente, en línea.

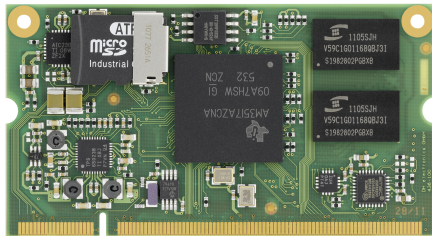


Figura: Embedded System.

Elementos principales

En su forma más general, un sistema embebido está constituido por:

- Unidad de procesamiento.
- Memoria.
- Entradas y salidas.
- Fuente de alimentación.
- *Software (Firmware).*

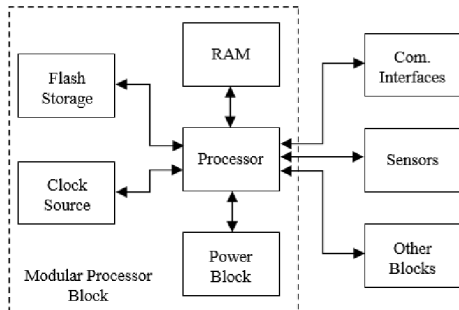


Figura: Diagrama en bloques de un sistema embebido.

Propiedades generales

Los sistemas embebidos presentan características que los distinguen de otros sistemas de cómputo.

- Función específica.
- Recursos limitados.
- Alta confiabilidad.
- Bajo consumo de energía.
- Integración entre hardware y software.

¿Dónde encontramos sistemas embebidos?

Presencia en la vida cotidiana

Actualmente los sistemas embebidos se encuentran presentes en prácticamente todos los sectores tecnológicos.

- Electrodomésticos.
- Vehículos.
- Equipos médicos.
- Automatización industrial.
- Telecomunicaciones.
- Internet de las Cosas (IoT).

Concepto fundamental

Un sistema embebido no es únicamente un microcontrolador. Es la integración de *hardware* y *software* para ejecutar una función específica de manera eficiente dentro de un sistema electrónico.

Origen de los sistemas embebidos

Primeras aplicaciones

Los primeros sistemas embebidos surgieron con el desarrollo de los circuitos integrados y los primeros microprocesadores durante la década de 1970. Su objetivo era automatizar funciones específicas dentro de equipos electrónicos.

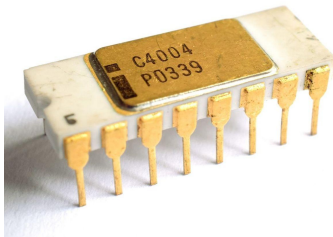


Figura: Intel arquitectura 4004

Los sistemas embebidos del presente

Los sistemas embebidos continúan evolucionando ofreciendo mayor capacidad de procesamiento y eficiencia energéticas.

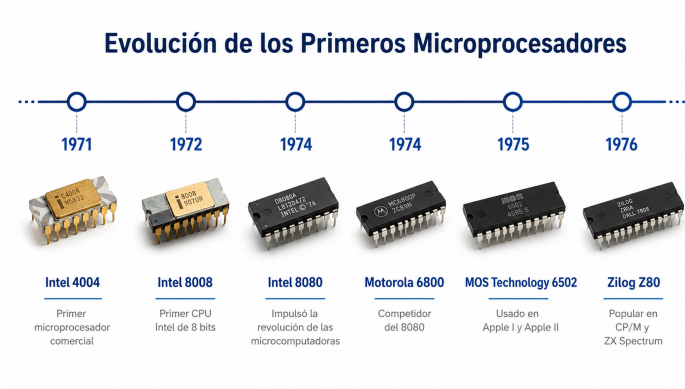


Figura: Línea de tiempo evolución de los primeros microprocesadores.

El papel del ingeniero

Retos futuros

El desarrollo de sistemas embebidos requiere profesionistas capaces de integrar conocimientos de electrónica, programación, comunicaciones y diseño de *hardware* para construir soluciones confiables y seguras.

Durante este curso

Las herramientas y metodologías estudiadas constituyen la base para el desarrollo de aplicaciones modernas utilizando microcontroladores.

A.3

Microprocesadores

¿Qué es un microprocesador?

Un microprocesador es un circuito integrado que contiene la unidad central de procesamiento. Su función principal es interpretar y ejecutar instrucciones, realizar operaciones aritméticas y lógicas, y coordinar el intercambio de información con la memoria y los dispositivos periféricos.

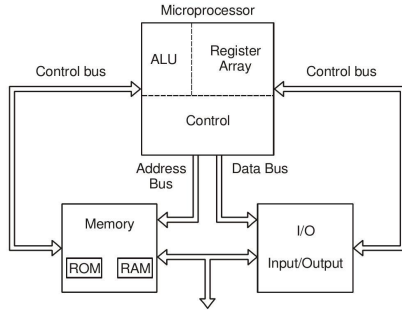


Figura: Arquitectura clásica de un sistema basado en microprocesador.

El microprocesador ejecuta instrucciones almacenadas en memoria mediante un ciclo continuo de operación.

- Obtiene la instrucción.
- La decodifica.
- Ejecuta la operación.
- Almacena o transmite el resultado.

El microprocesador dentro del sistema

Elementos externos necesarios

Un microprocesador normalmente requiere componentes externos para formar una computadora completa.

- Memoria de programa y de datos.
- Puertos de entrada y salida.
- Temporización y generación de reloj.
- Dispositivos periféricos.

Idea clave

El microprocesador representa principalmente la unidad de procesamiento; por sí solo no constituye un sistema completo.

Unidad Aritmético-Lógica (ALU)

La ALU es el bloque encargado de realizar las operaciones matemáticas y lógicas necesarias para ejecutar un programa.

- Sumas y restas.
- Operaciones lógicas.
- Comparaciones.
- Desplazamientos de bits.

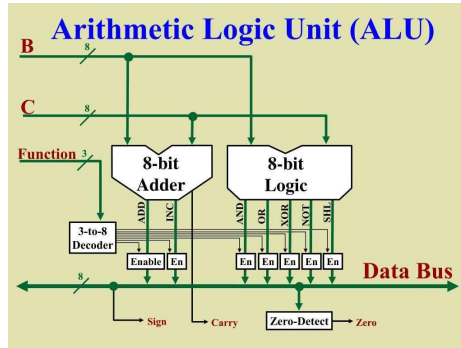


Figura: Diagrama interno de una ALU.

Unidad de Control y Registros

La Unidad de Control coordina la ejecución de las instrucciones, mientras que el banco de registros almacena temporalmente datos e información utilizada durante el procesamiento.

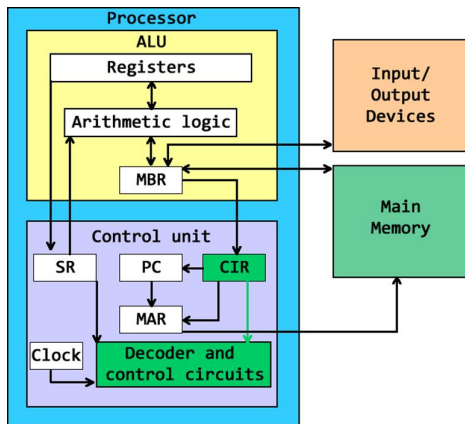


Figura: Interacción entre la Unidad de Control y el Banco de Registros

Buses del sistema

Los buses permiten el intercambio de información entre el microprocesador y el resto del sistema, estos son: bus de datos; bus de direcciones; y bus de control.

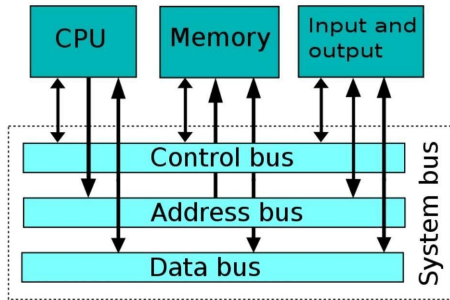


Figura: Bus de datos, direcciones y control.

Componentes fundamentales

Un microprocesador puede entenderse como la integración de cuatro bloques principales:

- Unidad de Control.
- Unidad Aritmético-Lógica.
- Banco de registros.
- Sistema de buses.

Estos elementos trabajan conjuntamente para ejecutar las instrucciones de un programa.

Ciclo de ejecución de instrucciones

El funcionamiento de un microprocesador consiste en ejecutar de forma secuencial las instrucciones almacenadas en memoria. Este proceso se conoce como ciclo de instrucción.

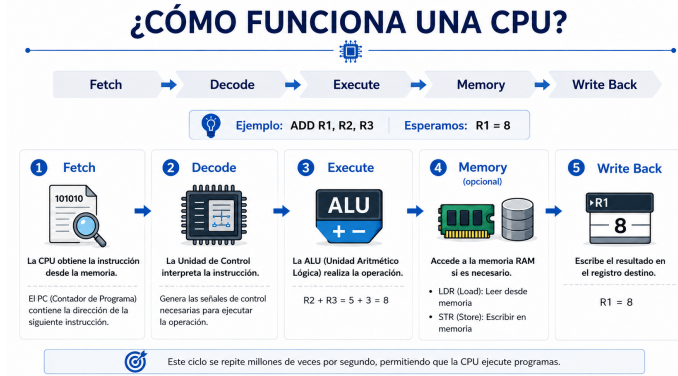


Figura: Diagrama del ciclo Fetch → Decode → Execute → Write Back.

Etapas del ciclo de instrucción

Proceso de ejecución

Cada instrucción sigue una secuencia básica de operaciones (Pipeline):

- ➊ Buscar la instrucción en memoria (Fetch).
- ➋ Decodificar la instrucción (Decode).
- ➌ Ejecutar la operación (Execute).
- ➍ Escribir el resultado (Write Back).

Nota: Se denomina *Pipeline* (tubería o canalización) porque, al igual que una línea de ensamblaje, mientras una instrucción se encuentra en una etapa, otras instrucciones pueden ocupar simultáneamente las etapas restantes. Esto permite aprovechar mejor los recursos del procesador y aumentar el número de instrucciones ejecutadas por unidad de tiempo.

Concepto fundamental

Todo programa, sin importar su complejidad, se ejecuta mediante la repetición continua del ciclo:

Buscar → Decodificar → Ejecutar → Actualizar

Comprender este ciclo facilitará el estudio de las arquitecturas de computadores y de los micro-controladores que se abordarán posteriormente.

Cuando una aplicación requiere ejecutar sistemas operativos completos, interfaces gráficas o múltiples procesos simultáneos, es común utilizar microprocesadores en lugar de microcontroladores.

- Raspberry Pi.
- BeagleBone.
- NVIDIA Jetson.
- Computadoras industriales.

¿Cuándo utilizar un microprocesador?

Aplicaciones recomendadas

Los microprocesadores son adecuados cuando la aplicación requiere:

- Sistemas operativos (Linux, Windows).
- Interfaces gráficas complejas.
- Gran capacidad de memoria.
- Procesamiento intensivo de datos.
- Ejecución simultánea de múltiples aplicaciones.

Idea principal

Los microprocesadores destacan por su elevada capacidad de procesamiento y flexibilidad, pero generalmente requieren memoria y periféricos externos para construir un sistema completo. En la siguiente sección se estudiará cómo los microcontroladores integran estos elementos dentro de un solo circuito integrado.

A.4

Microcontroladores

¿Qué es un microcontrolador?

Definición

Un microcontrolador es un circuito integrado diseñado para controlar dispositivos electrónicos. A diferencia de un microprocesador, integra en un solo chip la unidad de procesamiento, memoria y diversos periféricos.

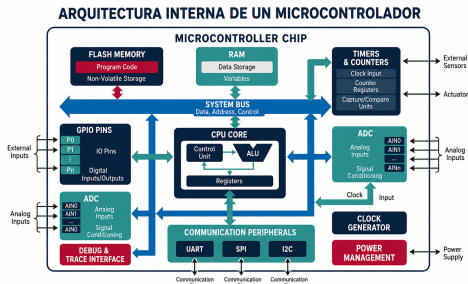


Figura: Arquitectura interna de un microcontrolador.

¿Por qué utilizar un microcontrolador?

Los microcontroladores integran los elementos necesarios para desarrollar aplicaciones embebidas.

- CPU.
- Memoria Flash y RAM.
- Periféricos de entrada y salida.
- Bajo consumo de energía.
- Tamaño reducido.

Raspberry Pi Pico

La plataforma utilizada durante el curso incorpora el microcontrolador **RP2040**, el cual integra dos núcleos ARM Cortex-M0+, memoria SRAM y una amplia variedad de periféricos programables mediante el Pico SDK.

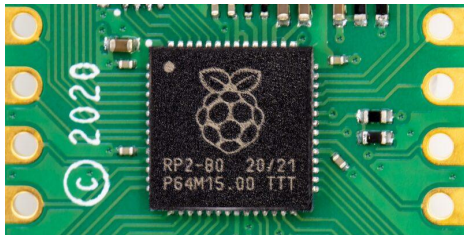


Figura: Microcontrolador RP2040 en encapsulado QFN-56.

Concepto fundamental

Un microcontrolador es un sistema de cómputo completo integrado en un solo circuito. Esta característica lo convierte en la solución ideal para la mayoría de las aplicaciones de control, monitoreo y automatización que se desarrollarán durante el curso.

Componentes principales

Un microcontrolador integra en un mismo circuito los elementos necesarios para implementar un sistema embebido.

- Unidad Central de Procesamiento (CPU).
- Memoria de programa y de datos.
- Periféricos.
- Sistema de reloj.

ARQUITECTURA INTERNA DE UN MICROCONTROLADOR

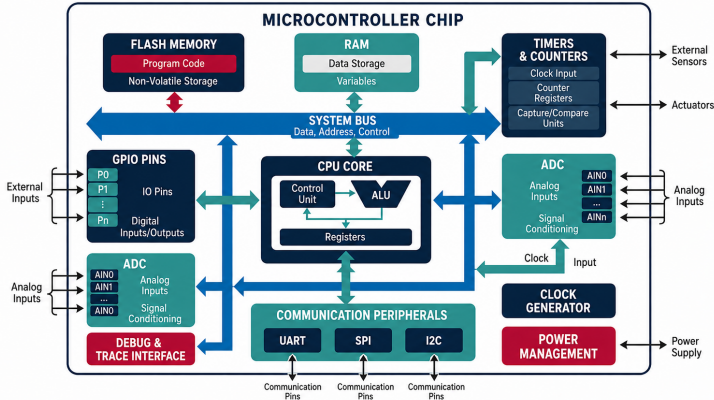


Figura: Arquitectura interna de un microcontrolador.

Componentes de un microcontrolador

Un microcontrolador moderno integra en un único circuito:

- CPU.
- Memoria.
- Periféricos.
- Sistema de reloj.

Esta integración permite desarrollar sistemas embebidos compactos, eficientes y de bajo costo.

Funcionamiento de un microcontrolador

Funcionamiento del microcontrolador

Un microcontrolador ejecuta un programa almacenado en memoria para supervisar y controlar un sistema electrónico. Durante su funcionamiento interactúa continuamente con sensores, actuadores y dispositivos de comunicación. Al igual que un microprocesador, la **CPU del microcontrolador ejecuta las instrucciones mediante un Pipeline.**

Entradas y salidas

El microcontrolador recibe información del entorno mediante sensores o dispositivos de entrada, procesa dichos datos y genera acciones sobre actuadores o dispositivos de salida.

- Lectura de sensores.
- Procesamiento de datos.
- Control de actuadores.
- Comunicación con otros dispositivos.

Actualmente existen diversos fabricantes de microcontroladores, cada uno orientado a diferentes aplicaciones, niveles de desempeño y costos.

- *Microchip Technology.*
- *STMicroelectronics.*
- *Texas Instruments.*
- *NXP Semiconductors.*
- *Infineon Technologies.*
- *Raspberry Pi Ltd.*
- *Espressif Systems.*

Ejemplos representativos

Cada fabricante ofrece diferentes familias de dispositivos adaptadas a necesidades específicas.

Fabricante	Familias
<i>Microchip</i>	PIC, AVR, SAM
<i>STMicroelectronics</i>	STM32
<i>Texas Instruments</i>	MSP430, Tiva C
<i>NXP</i>	LPC, Kinetis, i.MX RT
<i>Espressif</i>	ESP32, ESP8266
<i>Raspberry Pi Ltd.</i>	RP2040, RP2350

¿Por qué Raspberry Pi Pico?

Plataforma del curso

Durante este curso se utilizará la Raspberry Pi Pico debido a que ofrece una plataforma moderna, ampliamente documentada y adecuada para el aprendizaje de sistemas embebidos.

- Basada en el RP2040.
- Arquitectura ARM Cortex-M0+.
- Amplia disponibilidad.
- Documentación oficial abierta.
- Pico SDK gratuito.

A.5

Comparación entre microprocesadores y microcontroladores

Microprocesador vs. Microcontrolador

Microprocesador	Microcontrolador
CPU independiente.	CPU integrada.
Requiere memoria externa.	Memoria integrada.
Periféricos externos.	Periféricos integrados.
Mayor capacidad de procesamiento.	Optimizado para control.
Generalmente ejecuta un sistema operativo.	Ejecuta firmware dedicado.

Sistema basado en microprocesador

Un sistema basado en un microprocesador requiere diversos componentes externos para operar.

- Memoria RAM.
- Memoria de almacenamiento.
- Controladores de periféricos.
- Interfaces de comunicación.

Sistema basado en microcontrolador

En un microcontrolador la mayoría de los recursos necesarios para desarrollar una aplicación se encuentran integrados dentro del mismo circuito.

- CPU.
- Memoria Flash.
- SRAM.
- GPIO.
- Temporizadores.
- Interfaces de comunicación.

Comparación de aplicaciones

Microprocesadores	Microcontroladores
Computadoras personales	Electrodomésticos
Servidores	Sistemas de control
Interfaces gráficas	Automatización industrial
Procesamiento multimedia	Instrumentación electrónica
Inteligencia Artificial	Internet de las Cosas (IoT)

A.6

Arquitecturas de computadores

¿Qué es una arquitectura de computadores?

Definición

La **arquitectura de computadores** es el conjunto de principios que define cómo está organizado un sistema de cómputo y cómo interactúan sus componentes para ejecutar programas. Describe la forma en que el procesador, la memoria y los dispositivos de entrada/salida intercambian información.

¿Por qué es importante?

La arquitectura determina aspectos fundamentales del sistema, como:

- El rendimiento y la velocidad de ejecución.
- La organización de la memoria.
- La forma en que se ejecutan las instrucciones.
- El consumo de energía.
- El costo y la complejidad del hardware.

En esta sección aprenderás

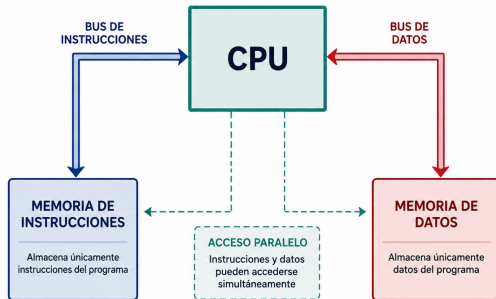
Se estudiarán las principales arquitecturas empleadas en microprocesadores y microcontroladores, comparando los modelos **Von Neumann** y **Harvard**, así como las filosofías de diseño **CISC** y **RISC**, ampliamente utilizadas en los sistemas embebidos modernos.

¿Qué es la arquitectura *Harvard*?

La arquitectura *Harvard* se caracteriza por utilizar memorias independientes para almacenar el programa y los datos.

ARQUITECTURA HARVARD

Memorias y buses separados para instrucciones y datos



CARACTERÍSTICAS

- Memorias separadas para instrucciones y datos.
- Buses independientes para instrucciones y datos.
- Permite acceder a instrucciones y datos al mismo tiempo.
- Mayor rendimiento.



VENTAJAS

- Mayor velocidad y ancho de banda.
- Acceso simultáneo a instrucciones y datos.



DESVENTAJAS

- Mayor complejidad y costo de hardware.
- Mayor consumo de recursos.

Figura: Arquitectura *Harvard* de un sistema de procesamiento.

Principales características

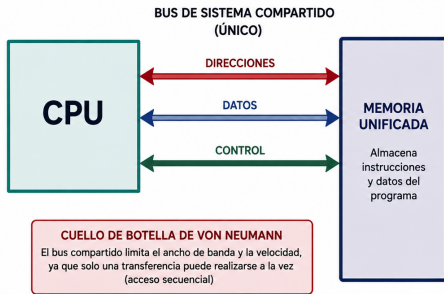
- Memorias independientes para instrucciones y datos.
- Buses separados para cada memoria.
- Acceso simultáneo a instrucciones y datos.
- Mayor velocidad de ejecución.
- Reduce el cuello de botella de memoria.

¿Qué es la arquitectura *Von Neumann*?

La arquitectura *Von Neumann* utiliza una única memoria para almacenar tanto las instrucciones del programa como los datos.

ARQUITECTURA VON NEUMANN

Memoria y bus compartidos para instrucciones y datos



CARACTERÍSTICAS

- Instrucciones y datos se almacenan en la misma memoria.
- Comparten el mismo bus para direcciones, datos y control.
- Solo una transferencia puede ocurrir a la vez (instrucción o datos).
- Diseño más simple y económico.

VENTAJAS

- Menor costo de hardware.
- Implementación más sencilla.

DESVENTAJAS

- Ancho de banda limitado por el bus compartido.
- Menor rendimiento en comparación con Harvard.

Figura: Arquitectura *Von Neumann* de un sistema de procesamiento.

Principales características

- Memoria compartida para instrucciones y datos.
- Un único bus de comunicación.
- Arquitectura sencilla.
- Menor complejidad de hardware.
- Posible cuello de botella en el acceso a memoria.

¿Qué es una arquitectura CISC?

La arquitectura **CISC** (*Complex Instruction Set Computer*) se caracteriza por disponer de un conjunto amplio de instrucciones, muchas de ellas capaces de realizar operaciones complejas..

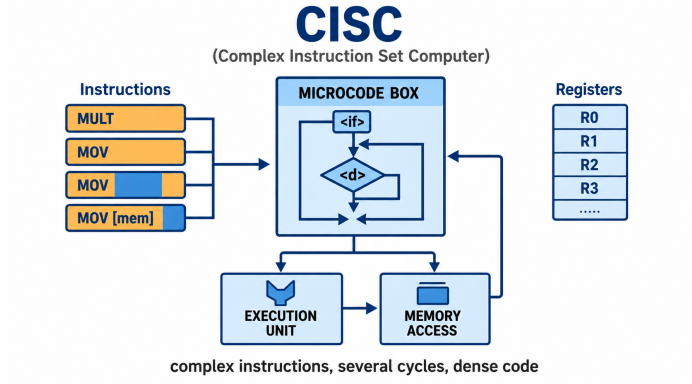


Figura: Arquitectura CISC (*Complex Instruction Set Computer*)

Principales características

Las arquitecturas CISC fueron diseñadas para reducir el número de instrucciones ejecutadas por un programa.

- Gran cantidad de instrucciones.
- Instrucciones de longitud variable.
- Diversos modos de direccionamiento.
- Mayor complejidad del *hardware*.
- Programas con menor número de instrucciones.

Procesadores representativos

Las arquitecturas CISC continúan siendo ampliamente utilizadas en computadoras personales y servidores.

- Intel x86.
- Intel x86-64.
- AMD64.

¿Qué es una arquitectura RISC?

La arquitectura **RISC** (*Reduced Instruction Set Computer*) se basa en un conjunto reducido de instrucciones simples y optimizadas para ejecutarse de forma eficiente.

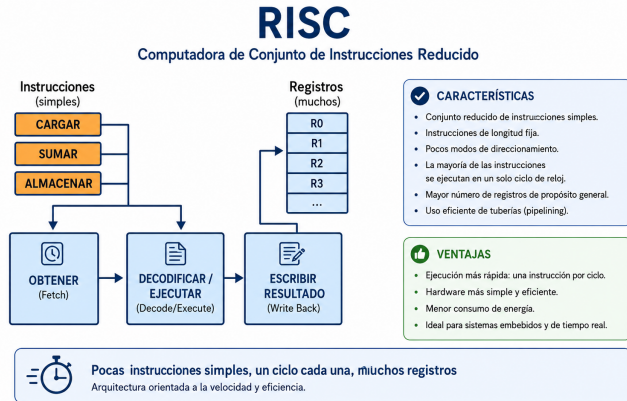


Figura: Arquitectura RISC (*Reduced Instruction Set Computer*)

Principales características

Las arquitecturas RISC están diseñadas para ejecutar instrucciones simples de manera rápida y eficiente.

- Conjunto reducido de instrucciones.
- Instrucciones de longitud fija.
- Gran número de registros.
- *Hardware* más simple.
- Alto rendimiento por ciclo de reloj.

Procesadores representativos

Actualmente, la filosofía RISC domina el mercado de los sistemas embebidos y dispositivos móviles.

- ARM Cortex-M (*Advanced RISC Machines*).
- ARM Cortex-A.
- RISC-V.
- MIPS.

Comparación CISC vs. RISC

Resumen

CISC

Instrucciones complejas.
Mayor complejidad del hardware.
Longitud variable de instrucciones.
Mayor consumo energético.
Computadoras personales y servidores.

RISC

Instrucciones simples.
Hardware simplificado.
Longitud fija de instrucciones.
Menor consumo energético.
Sistemas embebidos y dispositivos móviles.

Relación entre las arquitecturas

Dos decisiones de diseño diferentes

Al diseñar un procesador o microcontrolador deben responderse dos preguntas fundamentales:

① ¿Cómo se organiza la memoria?

- Arquitectura **Von Neumann**: memoria única para instrucciones y datos.
- Arquitectura **Harvard**: memorias y buses independientes para instrucciones y datos.

② ¿Cómo será el conjunto de instrucciones?

- Arquitectura **CISC**: instrucciones complejas y numerosas.
- Arquitectura **RISC**: instrucciones simples y optimizadas para ejecutarse rápidamente.

Idea clave

Las arquitecturas **Harvard** y **Von Neumann** describen la organización de la memoria, mientras que las arquitecturas **RISC** y **CISC** describen el diseño del conjunto de instrucciones. Son conceptos complementarios y pueden combinarse en un mismo procesador.

A.7

Raspberry Pi Pico

¿Qué es Raspberry Pi?

Raspberry Pi es una organización fundada en el Reino Unido con el propósito de promover la enseñanza de la informática y la programación mediante plataformas de bajo costo y fácil acceso.

En enero de 2021 Raspberry Pi presentó la Raspberry Pi Pico, una tarjeta de desarrollo basada en el microcontrolador RP2040, diseñado completamente por Raspberry Pi. Su objetivo fue ofrecer una plataforma moderna para aplicaciones de control, automatización y sistemas embebidos.

Características técnicas

- Arquitectura ARM Cortex-M0+ de 32 bits.
- Dos núcleos de procesamiento.
- Frecuencia de operación de hasta 133 MHz.
- 264 KB de memoria SRAM integrada.
- Soporte para memoria Flash externa mediante interfaz QSPI.

Interfaces disponibles

El RP2040 incorpora múltiples periféricos que permiten desarrollar una gran variedad de aplicaciones embebidas.

- GPIO.
- UART.
- SPI.
- I²C.
- PWM.
- ADC.
- PIO (Programmable I/O).
- Temporizadores.

Software utilizado durante el curso

- Visual Studio Code.
- ARM GNU Toolchain.
- CMake.
- Ninja.
- Pico SDK.
- Git.

Flujo de trabajo

El desarrollo de una aplicación sigue una secuencia de trabajo bien definida.

- 1 Escribir el código fuente.
- 2 Compilar el proyecto.
- 3 Generar el archivo ejecutable.
- 4 Programar la Raspberry Pi Pico.
- 5 Ejecutar y verificar el funcionamiento.

Laboratorio A

Reconocimiento de la Raspberry Pi Pico

Reconocimiento de la plataforma

El estudiante identificará los elementos físicos principales de la Raspberry Pi Pico, sus pines de conexión, opciones de alimentación, botón BOOTSEL y recursos básicos para el desarrollo de aplicaciones embebidas.

Material requerido

- Raspberry Pi Pico o Pico W.
- Cable USB para datos.
- Protoboard.
- LED y resistencia.
- Multímetro.
- Computadora con acceso a documentación técnica.

Durante la práctica se identificarán los principales elementos de la tarjeta.

- Microcontrolador RP2040.
- Puerto Micro-USB.
- Botón BOOTSEL.
- LED integrado.
- Pines GPIO.
- Pines de alimentación y tierra.

Pinout de la tarjeta

El estudiante deberá consultar el pinout oficial de Raspberry Pi Pico e identificar los pines asociados con alimentación, tierra, GPIO, ADC, UART, SPI, I²C y depuración SWD.

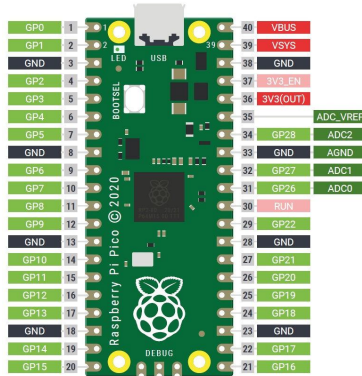


Figura: Pinout oficial de Raspberry Pi Pico

Modo BOOTSEL

Programación inicial

La Raspberry Pi Pico puede entrar en modo **BOOTSEL** manteniendo presionado el botón **BOOTSEL** mientras se conecta el cable USB al equipo.

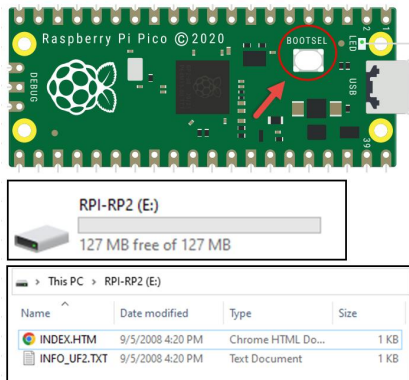


Figura: Ingreso al modo BOOTSEL

Desarrollo de la práctica

- 1 Identificar los componentes principales de la tarjeta.
- 2 Localizar los pines de alimentación y tierra.
- 3 Identificar GPIO, ADC, UART, SPI e I²C.
- 4 Verificar continuidad y alimentación con multímetro.
- 5 Entrar en modo BOOTSEL.
- 6 Registrar observaciones en la bitácora.

Evidencia de laboratorio

- Fotografía de la tarjeta utilizada.
- Tabla de pines identificados.
- Diagrama de conexión básico.
- Evidencia del modo BOOTSEL.
- Conclusiones técnicas de la práctica.

Resultado esperado

Al finalizar este laboratorio, el estudiante deberá reconocer físicamente la Raspberry Pi Pico y ubicar los recursos básicos que utilizará durante el resto del curso: alimentación, GPIO, comunicación, depuración y programación inicial.